

Legacy Analytics Project Documentation

1. Project Overview

Legacy Analytics is a browser-based React application that generates a deterministic life-factor analysis based on a user's date of birth. It calculates a parent-influence split, highlights which parent has the stronger overall influence, and presents the result in a polished dashboard with data tables, charts, summaries, and export options.

The project is designed to be:

- deterministic and explainable
- local-first and privacy-friendly
- responsive for desktop and smaller screens
- easy to validate with automated tests
- deployable as a static frontend without a backend

2. Repository and Deployment

- GitHub Repository: <https://github.com/prajakta-98/Legacy-Analytics>
- Live Deployment: <https://legacy-analytics.netlify.app>

3. Project Purpose

The application is based on a life-factor assessment model in which each date of birth produces a unique pattern of influence between Mother and Father. The calculator follows an assessment-inspired structure rather than claiming a scientific or medical formula. The purpose is to translate an input date into a stable, understandable score distribution that can be analyzed visually and exported for reporting.

The app intentionally emphasizes:

- transparency of logic
- deterministic values for the same date
- total reconciliation to 100.000
- no server-side data collection

4. What the Application Does

Users enter a valid date of birth, and the app:

1. validates the date in local format

2. checks if the date is empty, invalid, or future-dated
3. calculates the factor totals using a deterministic hashing algorithm
4. determines the dominant parent based on odd/even day rule
5. normalizes the total to exactly 100.000
6. displays a summary dashboard and factor breakdown
7. allows CSV and PDF export for reports
8. persists last-used values locally in the browser

5. Core Logic and Working Flow

5.1 Date Validation

The app validates the date before running calculations. It rejects invalid input and prevents impossible or future dates from being processed.

5.2 Hash-Based Deterministic Calculation

The calculation engine takes the ISO date string, hashes it, and uses that hash as a deterministic seed to generate factor variations. This ensures the same date always produces the same result while still producing varied final values across different dates.

The key idea is:

- each date has a stable hash
- the hash is used to create small variations around reference ranges
- final values are adjusted to maintain parent totals and overall normalization

5.3 Dominant Parent Rule

The project follows the assessment rule:

- odd day of month => Mother has the stronger overall influence
- even day of month => Father has the stronger overall influence

This rule is used to determine the dominant parent and shape the final total split.

5.4 Normalization to 100%

The final calculation ensures that:

- Mother + Father = 100.000
- all factor totals reconcile correctly

- each factor row can be expressed as mother + father = total
- the grand total is consistently normalized

This guarantees data integrity and makes the result consistent for the UI and exports.

5.5 Persistence and UX

The project stores the last valid date and result in browser storage so it behaves like a personal dashboard and preserves user context across refreshes.

6. Technology Stack

The app is built using a lightweight static frontend setup:

- React
- Vite
- JavaScript
- Recharts for chart visualization
- Lucide React for icons
- jsPDF + AutoTable for PDF export
- Vitest for testing

7. Features Added

7.1 DOB Input and Validation

- input field for date selection
- validation for empty dates
- invalid date rejection
- future-date blocking
- clear user-facing error messaging

7.2 Calculated Life-Factor Analysis

- seven life-factor categories
- individual mother/father contribution values
- total score per factor
- dominant parent label
- overall percentage difference indicator

7.3 Dashboard UI

- responsive summary cards
- aggregated statistics section
- comparison table with factor breakdown
- grouped visualization charts
- insight panel for interpreted findings

7.4 Theme Support

- dark mode by default
- light mode toggle
- accessible contrast and clean visual design

7.5 Export Functionality

- CSV export for report data
- PDF export for printable documents

7.6 Local Persistence

- saving last valid date and latest calculation
- browser-local state retention

7.7 Test Coverage

- tests for date validity
- determinism checks
- boundary conditions
- leap-year handling
- total invariant validations

8. Application Structure

The project is organized in a clean modular structure:

- `src/App.jsx` – main application shell
- `src/pages/Analytics.jsx` – dashboard logic and layout
- `src/hooks/useLifeFactorCalculator.js` – date validation and calculation lifecycle
- `src/utils/calculationEngine.js` – core formula and invariant checks
- `src/data/lifeFactors.js` – factor ranges and reference data
- `src/components/` – reusable UI parts for summary, table, charts, sidebar, and header

- src/utils/ – export, formatting, date, storage, and validation logic

9. Important Design Decisions

9.1 Client-Side Only

No backend or database is required. This makes the project simple, fast, and secure for a personal or educational dashboard.

9.2 Deterministic Calculation

The application deliberately avoids arbitrary randomness. It uses a stable algorithm so the same input date always returns the same output, making it suitable for testing and consistent auditing.

9.3 Invariant Validation

The engine validates the result before rendering it. It checks that:

- parent totals sum correctly
- grand total equals 100
- factor totals reconcile
- values are within valid ranges
- the dominant parent matches the input date rule

This protects the UI and prevents malformed calculations from appearing.

10. Strengths of the Project

- clear modular structure
- good separation of concerns
- deterministic calculation model
- strong user-facing analytics layout
- local export support
- strong test confidence
- easy deployment on static hosting

11. Limitations and Considerations

The project is not a medical or scientific diagnostic tool. It is a deterministic assessment-inspired app built around a workbook-style reference model. Some areas that could be improved include:

- more detailed explanations for each factor

- language personalization for users
- improved chart interaction
- stronger accessibility audit
- better onboarding or tutorial flow

12. Future Enhancements and Additions

12.1 User Experience Improvements

- onboarding screen for first-time users
- step-by-step guided explanation of each factor
- tooltips and help text for every metric
- more responsive mobile-first redesign

12.2 Analytics Enhancements

- compare multiple birth dates side by side
- save multiple historical calculations
- trend analysis across previous entries
- advanced filtering and sorting in the report table

12.3 Additional Export Features

- PNG and JPG report snapshots
- HTML report generation
- Excel export
- shareable summary cards

12.4 Personalization and AI Features

- AI-generated interpretation of results
- recommendations based on dominant factors
- personalized insights page
- multilingual support

12.5 Backend and Data Expansion

- user accounts
- cloud backup of results
- remote database to store historical reports

- authentication and protected dashboards

12.6 Professionalization

- custom branding and logo
- landing page with marketing content
- dark/light theme designer settings
- domain and SEO optimization

13. Suggested Roadmap

Phase 1: Stabilize and polish

- improve accessibility
- finalize onboarding flow
- optimize mobile layout
- refine UX copy

Phase 2: Add intelligence

- factor explanations
- personalized insights
- side-by-side comparisons
- export improvements

Phase 3: Productization

- user authentication
- cloud storage
- analytics dashboard for multiple users
- branded deployment

14. Final Assessment

Legacy Analytics is a well-structured, modern frontend project that combines assessment logic, interactive statistics, compelling visualization, and export features in a single static app. It demonstrates a thoughtful balance between deterministic logic and user-friendly presentation. The project is already in a strong state for a portfolio, demo, or MVP, and it has clear opportunities for growth into a more complete product.

15. Conclusion

This project shows how a simple input like a date of birth can be transformed into a meaningful, structured comparative analysis. It is a good example of a deterministic UI-driven analytics tool that maintains trust, clarity, and ease of use without depending on a backend.

The combination of algorithmic logic, dashboard reporting, and export features makes it a strong candidate for further enhancement, feature expansion, and product-level refinement.